Semestral work assignment number 14
Fast Insertion Based Optimization of Bounding Volume Hierarchies

Michal Sladek[1]
Department of computer graphics and interaction,
Electrotechnical faculty, CTU Prague

Abstract

Implement an algorithm for optimizing BVH using insertion-based optimization (removal of nodes + insertion at better positions). First, construct the BVH top-down method using spatial medial and then use the optimization. Report the BVH cost before and after the optimization and the times for the optimization. Test the implementation on scenes (consisting of triangles) with a different count of objects and report the performance for at least primary and shadow rays (point light sources). Platform: C++, Windows/Linux

Keywords: ray tracing, bounding volume hierarchy, optimization

1. Introduction

This semestral work focuses on the implementation of an algorithm that was proposed in the paper *Fast Insertion Based Optimization of Bounding Volume Hierarchies* [1]. The BVH is first built using the top-down spatial median method and then optimized using this algorithm.

When using ray tracing for the rendering of real-time interactive applications the rendering speed is crucial. Even a relatively small improvement to the rendering time is able to increase the number of frames that can be rendered per second, which increases the interactivity of the application. Ray tracing can be sped up by using data structures to organize the primitives in the scene, which allows for finding ray-primitive intersections much faster. This means that building a high quality data structure is very important.

The goal of this work is to implement the proposed algorithm and measure statistical data on multiple scenes to be able to analyze the efficiency of this method and compare the results to the original paper.

2. Algorithm Description

The algorithm applies 2 optimizations to the BVH. The first one is the optimization loop and the second is tree compaction (both as proposed in the original paper). The node and tree cost are calculated using the the formulas from the original paper. Pseudocode for the overarching optimization function can be seen in Algorithm 1.

[1] B4M39DPG – Michal Sladek, winter semester 2024/25

2.1. Optimization loop

The optimization loop applies the optimization algorithm in passes. Each pass of the algorithm updates the top k of the tree's most inefficient nodes (k used was 1%). A node's inefficiency value is calculated using the M_{COMB} heuristic from the original paper.

Then, each of the inefficient nodes are removed and re-inserted into the tree (one by one). When a node N is removed, it's children (L and R) and parent (P) nodes are also removed and separated. N and P are both inner nodes and L and R are both sub-trees (inner or leaf nodes).

The order in which L and R are re-inserted is based on which of the two's bounding boxes has a larger surface area. In order to reinsert these sub-trees (let's say L is currently being re-inserted) into the BVH, a new optimal position for it has to be found - a node X that minimizes the increase in sum of *direct*[2] and *induced*[3] costs if merged with L. This is done using a branch and bound algorithm that allows for efficient search by pruning the branches that wouldn't lead to finding the optimal node X. Once such a node X is found, L is merged with it using either P or N as their parent node. The same process repeats for the other node.

It is also important to note that removing or re-inserting a node N from/to the tree means that all of the parent inner node bounding boxes have to be recalculated all the way up to the root node of the tree. Another thing that has to change with every removal and re-insertion is the inner node flag. This also applies to all of the parent nodes from N all the way up to the root node.

How the tree cost changes with every pass of this loop can be seen in Figure 3.

2.2. Terminating the optimization loop

After each pass of the optimization loop the new tree cost is saved into a list. As per the original paper, the optimization loop ends if the cost of the tree hasn't improved within a certain number of passes p_T. That is, if the tree cost in pass p (latest value) is larger than the tree cost in pass $p - p_T$ (older value) the optimization terminates.

2.3. Tree compaction

The BVH construction method creates a BVH that has exactly 1 primitive in each leaf node. It is usually more beneficial to store more primitives per leaf (e.g. 8-10) and so the tree is traversed again and compacted. If the cost of an inner node is larger than the added costs of the leaf nodes in it's left and right sub-trees, the inner node is compacted into a single leaf node that contains all of the sub-tree's leaf node primitives. This allows the tree cost to decrease even further.

3. Implementation details

The total time needed for this semestral work was around 70 hours. I spent the most time on trying to increase the performance of the optimization functions.

[2]Surface area of the bounding box that is the union of the bounding boxes of L and X.

[3]The accumulated increase of the surface area on the path from the root to the parent of X.

```
begin
    push_front (treeCost of the tree) to costs list;
    while true do
        create empty set inefficientNodes;
        FindInefficientNodes (inefficientNodes);
        foreach node in inefficientNodes do
            create empty list removedNodes;
            RemoveNode (node, removedNodes);
            if removedNodes is not empty then
                ReinsertNode (removedNodes.front);
                ReinsertNode (removedNodes.back);
            end
        end
        newTreeCost = GetTreeCost;
        push_back (newTreeCost of the tree) to costs list;
        if costs.size = p_T then
            if costs.front > costs.back then
                break;
            end
            pop_back() of costs list;
        end
    end
    CompactTree();
end
```

Algorithm 1: Optimization method.

3.1. Primitives per node

While trying to figure out why none of the scenes were showing any meaningful progress in ray tracing time with the optimization on, I noticed that one scene rendered especially slowly, the *hairball* scene. This was caused by the fact that the number of primitives per leaf node wasn't limited. During tree compaction the leaf nodes reached up to 1922 primitives per leaf. I ended up limiting the number of primitives per leaf to 8. This change improved the rendering time, but only slightly, so the search for the real cause of the problem continued.

3.2. BVH inner node flags

The real cause of the aforementioned slow ray tracing ended up being the inner node flags. These flags play an important role in the traversal of the tree and are initially set when building the tree top-down. Based on a node's flag, the traversal algorithm determines which sub-tree should be traversed (left, right or both). Not changing the flag of the nodes affected by the tree topology changes resulted in each ray needing a lot more traversal steps through the tree while searching for an intersection. Fixing this brought the ray trace time close to the time without optimization.

3.3. Inefficient node search

While generating the tables for this document, one thing stood out. At first glance the BVH tree data seemed correct, but the number of traversal steps and incidence operations per ray was barely better than without the optimization. After generating a graph that plotted the tree cost based on the number of passes of the optimization. I noticed that after a few

passes the tree cost stopped changing as opposed to the expected behavior - oscillation around the minimal tree cost. While rewriting the *FindInefficientNodes()* function from a recursive to a queue-based approach, I didn't change *return;* to *continue;* in the leaf node branch. This meant that the inefficient node list was just filled with nodes along a single path towards the left-most leaf node. After a few passes all of those nodes were optimized and the tree cost stopped changing.

One more issue that arose here was that the inefficiency value for some nodes was *inf*. The inefficiency value is calculated using the surface area of the current node and the child nodes. In one case an inner node's child node was a leaf that contained a triangle whose vertices were aligned along the x axis, forming a 1D line. Such a triangle's bounding box is also a line, and as such, the surface area is 0. Plugging this into the node inefficiency heuristic resulted in division by 0. Instead of resulting in error, this just assigns the value *inf* to the evaluated expression, which I wasn't aware of. I ended up resolving this issue by making the bounding box always return at least some tiny value as the surface area.

3.4. Shadow rays

When testing the shadow ray implementation I ran into the usual problem. Objects with dark pixels scattered over them due to floating number errors. This was fixed quickly by simply adding an epsilon to the ray-triangle intersection test that makes it ignore any triangle intersections that are too close to the ray origin. This epsilon value has to be changed individually for each model due to their size. While the bounding box of the *Armadillo* scene is roughly 130,000 units across in every axis, the bounding box for the *dragon* scene is not even 1 unit long along it's longest axis. This value is defined in the *triangle.h* file as *INTERSECTION_EPSILON*.

When rendering the architectural scenes another problem surfaces. As can be seen in Figure 1, it almost appears as if whole triangles were missing instead of just pixels like in the case of the single object scenes. The issue only gets worse after applying the optimization algorithm. Initially I thought that it could be because of the inner node traversal flags, so I triple checked those and it appeared they are all correct. There are also these strange squares missing in both of the pictures. After debugging I found out that the rays in those locations have the correct points of origin and the correct direction. The only strange part is that they found an intersection with an object, but not one that is close and can be mitigated by using the aforementioned epsilon value. Of course increasing the epsilon value helps, but not by the time the desired shadows start getting removed near objects that cast them. Next I thought it could also be caused by wrong normal orientation, but the only wrong normals in the *fairyforest* scene are located inside of the tree (which is why the shadow doesn't render there). I also noticed that the missing pixels don't correspond to object triangles and tried removing some background walls to see if they cause the missing squares, but they don't.

After laying out the data in Blender, I found out that the ray origin and direction are both correct, however in some cases (the missing pixels) an intersection is found between the ray origin and the light source that isn't there. I thought it was caused by issues with the BVH, but in that case, the primary rays would also be incorrect and the issues wouldn't show on the images where optimization wasn't applied. I ended up finding no pattern in the errors and no solution. In the end, this only affects the rendered image and doesn't affect the performance

of the BVH (all of the shadow rays are still casted and the BVH is traversed), so it has no effect on the collected data.

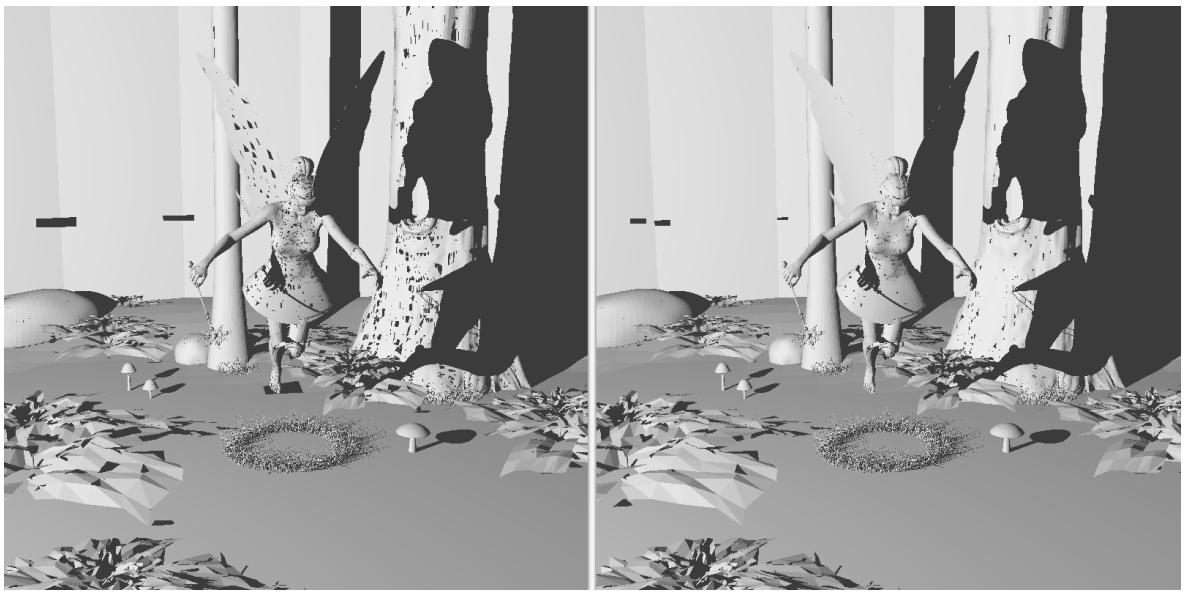

Figure 1: Comparison of two images of the *fairyforest* scene generated with shadow rays and optimization OFF (right) and ON (left). No epsilon value was applied in either picture.

3.5. Speed optimization

Once the aforementioned issue with finding inefficient nodes was removed the algorithm started behaving as expected, however the speed became an issue. Comparing to the original paper using the *powerplant* scene, the build time listed there was roughly the same as the time to to optimize the BVH. In the case of my implementation, the build time was measured at just over 8s and the optimization took over 85s. Although different building algorithms were used in these implementations, the time still gives a pretty good rough idea as to what kind of optimization time I'm looking for.

Firstly testing using the built in timer, I noticed a few things:

- From the point of calling the Optimize function where the timer starts, to being in the body of the Optimize function takes $\tilde{0}$.6s.
- The optimization took 20 passes to terminate. Each pass consisted of finding the inefficient nodes ($\tilde{1}$s) and then removing them and re-inserting their children back into the tree ($\tilde{3}$.3s).
- The tree compaction took $\tilde{2}$.5s.

I attempted to speed up the inefficient node search by only recursing into the node's children if that node was added into the list of inefficient nodes. This halved the time of the search but on the other hand, it wasn't finding all of the most inefficient nodes and each pass of the optimization lowered the cost slightly less when compared to the previous approach. I

also rewrote the function to a queue-based BFS search, but the same issues persisted. I ended up discarding these changes in favor of better functionality. Perhaps creating an array of node pointer and inefficiency measure pairs and using the quickselect algorithm would speed the process up.

The only problem I identified with the node removal and re-insertion algorithm is that while searching for the re-insertion node the priority queue order is based on the inverse value of the induced cost. In my implementation it is based on the actual value of the induced cost. Changing this resulted in no speed increase. While comparing the average number of nodes traversed for both of these methods I found no difference and both methods seemed to be pruning the search very efficiently.

The compaction function also isn't very efficient because the compacted leaf nodes are traversed twice. However in the context of the whole optimization time this wasn't the most pressing issue, so I focused on the other functions.

Using the profiler and compiling with the -O2 flag, the top 5 functions that take the most time are:

- 20.0701% - *ReinsertNode()*
- 17.3503% - *FindInefficientNodes()*
- 12.5059% - *adjust_heap()* function for a priority queue
- 12.0691% - *CTDBVH::SBVHleaf::GetBox()*
- 10.8130% - another priority queue related function

It seems that the *FindNodeForReinsertion()* function was optimized out, which is why it's parent function *ReinsertNode()* is seeing so much use. Furthermore, the priority queue functions also belong to the *FindNodeForReinsertion()* function. Rewriting the comparator lambda to a function had no effect on the performance.

4. Results

All of the testing was done on a 64-bit Ubuntu Linux machine with a 6-core Intel Core i7-9750H CPU (2.60 GHz base frequency, 12MB cache) and 16 GB of RAM. The implementation is a single-threaded application that runs on the CPU and it is compiled using *gcc* version 13.3.0.

The optimization method was tested on 9 different scenes (obtained from [2] and [3]), which can be seen in Figure 2. Five of these scenes (*A10*, *Armadillo*, *buddha*, *dragon* and *hairball*) are individual objects, while the other four (*fairyforest*, *powerplant*, *sibenik* and *sponza*) are architectural scenes. Architectural scenes are typically larger scenes that consist of many objects.

Table 1 lists and explains all of the measurements that were taken during testing and that are used in Tables 2 and 3 for the evaluation of the results.

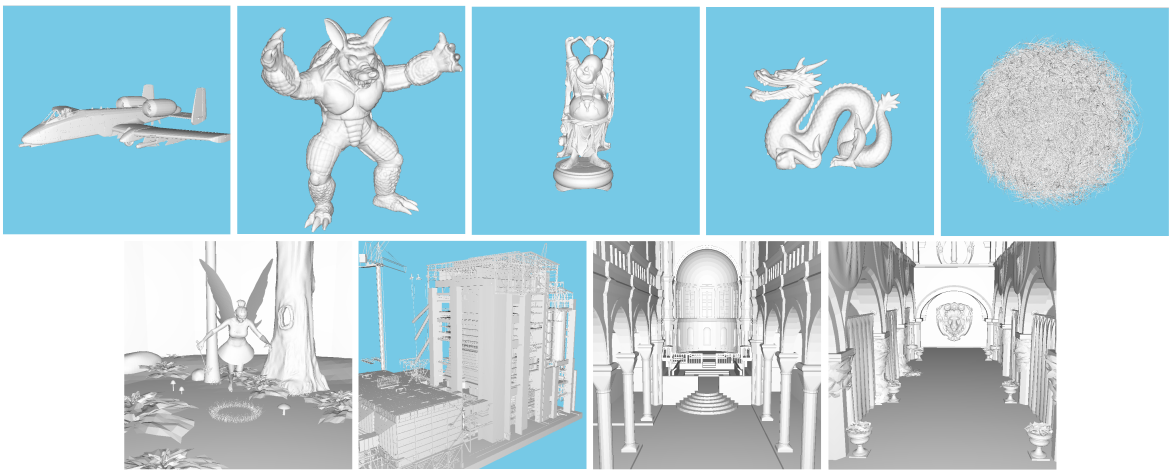

Figure 2: Renders produced by primary ray casting of the different scenes. Top row contains the individual object scenes: *A10*, *Armadillo*, *buddha*, *dragon* and *hairball*. The bottom row contains architectural scenes: *fairyforest*, *powerplant*, *sibenik* and *sponza*.

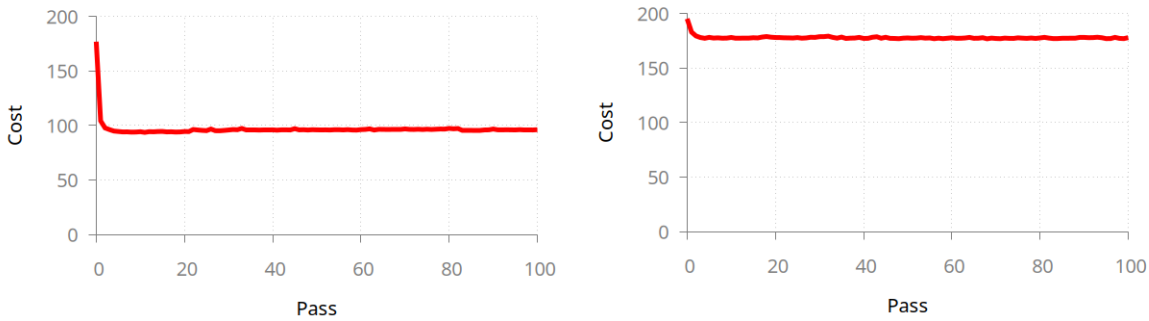

Figure 3: Graphs showing how the BVH tree cost lowers with every pass of the optimization loop. Graphs are made for the *powerplant* scene (left) and the *buddha* scene (right).

4.1. Table 2: BVH statistics

Table 2 focuses on the structure of the BVH before and after optimization. As mentioned in the previous section, the optimization algorithm is not optimal. T_U is $\tilde{1}$0-20x slower than it should be when compared to the ratio of T_B and T_U (when the BVH is built using spatial median) from the original paper, where T_U is roughly the same as T_B.

The tree costs are different between my results and the original paper. The comparison only applies to 6 of the 9 scenes due to different triangle counts or missing objects. Three of the scenes (*buddha*, *dragon*, *fairyforest*) have comparable tree costs, while the other 3 have very different tree costs. The biggest outlier is the *hairball* scene (my cost: 776.34, original paper cost: 1003.56). I speculate 3 reasons for these differences:

- The original algorithm also applies random sampling of nodes to optimize the tree cost further after the M_{COMB} heurisic search.

abbreviation	meaning
T_B	Time to build the BVH
T_U	Time to optimize the BVH
opt.	If the BVH optimization was on or off
cost	Cost of the tree
N_I	Number of inner nodes in the BVH
N_L	Number of leaf nodes in the BVH
D_AVG	Average depth of a leaf node in the BVH
D_MAX	Maximum depth of a leaf node in the BVH
N_AP	Average number of primitives per leaf node
time	Time to ray trace the image
T_R	Average time per ray
N_IT	Average number of incidence operation per ray
N_TR	Average number of traversal steps per ray
PERF	Number of rays cast per second
N_Q	Total number of ray casts

Table 1: Abbreviations used in the following tables and their meaning.

- The original paper doesn't specify the maximum number of primitives per leaf during the tree compaction. Most of the cost reduction of the *hairball* scene happens in this stage. If I set the maximum number of primitives per node to 3, the resulting cost is 1024.6, which is very close to 1003.56 of the original paper, however this change also increases the ray tracing time by about 15%.
- My implementation iterates over splitting axis during BVH construction using the round robin method. Perhaps the original paper uses a different method (such as splitting along the longest axis) which could result in a different data structure.

Otherwise, the N_I and N_L are reduced and N_AP is increased in every scene thanks to the tree compaction. In most scenes, D_AVG and D_MAX also decrease, but in some it increases slightly. The optimization algorithm will re-insert a node at a position that minimizes it's direct and induced cost, even if this means that the depth of the tree will increase. Tree compaction brings the depth of the tree down again, but sometimes the resulting depth is still larger than in the case of an unoptimized BVH. All of this is expected behavior.

4.2. Table 3: Ray tracing statistics

Optimization on all of the architectural scenes shows ray tracing time reduction of up to 65.46% (*sponza* scene with shadow rays on).

On the other hand, the individual object scenes only show a slight increase in ray tracing performance, while the *Armadillo* scene even shows a decrease. *Hairball*, *dragon* and *A10* scenes show an increase, however, based on the number of meshes that makes up the *A10* scene, it should categorize more as an architectural scene.

At first I thought that the more objects a scene has, the better optimized BVH ray tracing performance. However, if that were the case, the *fairyforest* scene should have seen a bigger increase in performance than it did. I think that the performance increase is due to the distribution of the triangles in the scene. The larger scenes (*powerplant* or *sponza*) seem to have

scene [*triangles*]	T_B [*s*]	T_U [*s*]	opt. [*on/off*]	cost	N_I	N_L	D_AVG	D_MAX	N_AP
A10	0.089	0.928	off	89.33	218,651	218,652	25.491	42	1.000
218,652			on	44.74	110,227	110,228	26.681	46	1.984
Armadillo	0.142	1.144	off	99.14	345,943	345,944	19.681	27	1.000
345,944			on	83.96	171,870	171,871	18.348	25	2.013
buddha	0.441	3.950	off	195.31	1,087,473	1,087,474	21.620	35	1.000
1,087,474			on	159.78	542,235	542,236	20.389	35	2.005
dragon	0.396	3.231	off	173.42	871,305	871,306	21.274	36	1.000
871,306			on	142.78	437,885	437,886	19.864	35	1.990
fairyforest	0.067	0.382	off	111.15	174,116	174,117	24.431	43	1.000
174,117			on	71.04	86,308	86,309	24.190	40	2.017
hairball	1.132	20.894	off	1580.70	2,879,999	2,880,000	25.518	1,437	1.000
2,880,000			on	1024.60	1,362,242	1,362,243	24.716	1,435	2.114
powerplant	5.191	85.512	off	176.93	12,759,245	12,759,246	33.688	52	1.000
12,759,246			on	64.51	6,758,403	6,758,404	34.492	48	1.888
sibenik	0.026	0.282	off	193.07	75,283	75,284	19.239	29	1.000
75,284			on	112.90	39,189	39,190	18.983	30	1.921
sponza	0.102	1.615	off	407.91	262,266	262,267	23.507	37	1.000
262,267			on	143.93	136,111	136,112	24.979	36	1.927

Table 2: A table showing all of the BVH tree statistics for all of the scenes with optimization off and on.

a better distribution of triangles in the scene (and they take up more of the screen). Where as the scenes with a single mesh have a worse distribution of triangles (they are all mostly bunched up together in a smaller part of the screen). This would also explain why the *fairy-forest* scene only shows a small increase in performance. While it has a lot of objects, most of the scene's triangles are all bundled up together, forming the fairy or the tree trunk behind her.

I definitely expected a bigger increase in performance, especially in the case of individual objects. In the original paper, the performance increase achieved was around 40% and in the architectural scenes it was 80-95%.

5. Conclusion

Overall, I'm frustrated with my inability to find the issues causing the incorrect shadow ray casting and the sources of the poor performance. On the other hand I'm happy that the results show that the algorithm is on the right track and that the optimization is doing at least some of what it's supposed to do. The implementation also proved to be a very good way of 'playing' around with the data structure instead of just learning about it.

Creating batch scripts to run the application, collect data and rearrange it into LaTeX tables or graphs, while time consuming, is a very good way of debugging the application and finding patterns (that would otherwise be hard to notice) in the results it generates. This way of debugging can provide invaluable insight into the run of the application and perhaps even show where potential fixes need to be made.

If I had to implement this semestral work from the ground up again, I would have definitely started with creating a plan based on reading the article multiple times as well as the assignment. I think the first implementation step would have been shadow ray casting to rule out the possibility that the issues were caused by the tree optimization. Next step would have been to implement statistics data collection and interpretation into an easily readable form. This would have helped greatly with the optimization algorithm debugging later. Then I would have definitely spent more time to understand the intricacies of the code base I was working with (*nanogolem*) as well as the data structure I had to work with. And finally, the optimization algorithm implementation itself.

Most importantly I would have given myself more time. Without understanding the scope of the assignment well I decided to leave the implementation to when I started feeling better after dealing with illness and after the presentation on BVH(which I couldn't even attend due to said illness). While the implementation itself is rather simple, I underestimated how many problems I would encounter along the way and how much time it would take.

References

[1] Jiri Bittner, Michal Hapala, Vlastimil Havran. Fast Insertion-Based Optimization of Bounding Volume Hierarchies LaTeX, In Computer Graphics Forum, 2013. `https://onlinelibrary.wiley.com/doi/full/10.1111/cgf.12000`

[2] Computer Graphics Archive. Morgan McGuire. `https://casual-effects.com/data/`. Page from the year 2017.

[3] The Utah 3D Animation Repository. Ingo Wald. `https://www.sci.utah.edu/~wald/animrep/`.

scene [*triangles*] [*meshes*]	opt. [*on/off*]	time [*s*]	T_R [μs]	N_IT [*per ray*]	N_TR [*per ray*]	PERF [*kilorays/s*]	N_Q [*rays*]	rays calculated
A10 218,652 780	off	0.168	0.262	16.58	14.56	3,809.50	640,000	primary
	on	0.133	0.208	11.70	9.18	4,812.00		
	off	0.223	0.318	21.35	18.88	3,145.70	701,486	primary + shadow
	on	0.179	0.255	15.32	12.17	3,918.90		
Armadillo 345,944 1	off	0.330	0.516	32.71	30.01	1,939.40	640,000	primary
	on	0.408	0.637	32.23	27.55	1,568.60		
	off	0.577	0.716	41.36	37.67	1,397.00	806,043	primary + shadow
	on	0.616	0.764	41.34	34.90	1,308.50		
buddha 1,087,474 1	off	0.179	0.280	12.86	11.12	3,575.40	640,000	primary
	on	0.163	0.255	12.13	9.68	3,926.40		
	off	0.309	0.433	21.06	18.50	2,307.90	713,126	primary + shadow
	on	0.308	0.432	20.01	16.08	2,315.30		
dragon 871,306 1	off	0.269	0.420	18.92	16.91	2,379.20	640,000	primary
	on	0.238	0.372	17.96	14.84	2,689.10		
	off	0.378	0.506	28.19	25.30	1,977.40	747,470	primary + shadow
	on	0.382	0.511	27.06	22.33	1,956.70		
fairyforest 174,117 1,789	off	0.744	1.163	84.23	76.18	860.22	640,000	primary
	on	0.616	0.962	70.62	58.76	1,039.00		
	off	1.637	1.303	94.72	83.97	767.32	1,256,101	primary + shadow
	on	1.679	1.337	80.20	64.84	748.12		
hairball 2,880,000 1	off	1.632	2.550	124.93	104.56	392.16	640,000	primary
	on	1.418	2.216	110.84	86.25	451.34		
	off	2.099	2.506	123.55	103.11	399.08	837,662	primary + shadow
	on	1.921	2.293	110.70	85.82	436.06		
powerplant 12,759,246 57	off	2.095	3.273	221.18	204.19	305.49	640,000	primary
	on	0.911	1.423	93.08	76.07	702.52		
	off	3.376	3.737	230.61	213.77	267.56	903,276	primary + shadow
	on	1.378	1.526	98.00	80.67	655.50		
sibenik 75,284 1,087	off	0.961	1.502	130.29	115.12	665.97	640,000	primary
	on	0.712	1.112	96.07	76.14	898.88		
	off	1.683	1.491	125.32	112.45	670.52	1,128,489	primary + shadow
	on	1.344	1.191	90.57	73.12	839.65		
sponza 262,267 393	off	2.189	3.420	285.07	259.33	292.37	640,000	primary
	on	0.756	1.181	85.41	69.53	846.56		
	off	4.447	3.571	292.59	265.39	280.02	1,245,229	primary + shadow
	on	1.890	1.518	102.86	83.45	658.85		

Table 3: A table showing all of the ray tracing statistics for all of the scenes, with BVH optimization on and off, and with shadow ray casting on and off. Note that N_Q is always 640,000 for primary ray casting because the size of the rendered image is 800x800 pixels.